

# Hashing con Linear Probing

Con il **concatenamento separato**, ciascuno degli  $n$  bucket contiene una **lista concatenata** (“catena”) di chiavi che sono state mappate in quel bucket.

Alcuni **svantaggi pratici**:

la memorizzazione di tutti quei **puntatori** richiede molta memoria;  
e, cosa più importante, **non è cache-friendly**, perché gli elementi di una stessa lista di bucket sono in genere **sparsi in memoria**.

Una soluzione di hashing più pratica, detta **bucket hashing con linear probing**, **non richiede puntatori** ed è **più efficiente per la cache**.

# Hashing con Linear Probing

L'idea di base del **linear probing** è di memorizzare **una sola chiave per cella** dell'array  $B$ .

Se più chiavi hanno lo **stesso valore di hash**, esse vengono inserite nella **prima cella libera successiva** nell'array  $B$ .

In questo modo, durante la **ricerca** di una chiave si leggono **celle consecutive** dell'array, che di norma si trovano nella **stessa linea di cache** — rendendo l'accesso molto più efficiente.

# Hashing con Linear Probing

Il **linear probing** utilizza un **array**  $B$  di dimensione  $n > m$ , dove  $m$  è il numero di elementi memorizzati.



array  $B$

Tipicamente, quando si usa il linear probing, vale  $n > 2m$ , cioè il **fattore di carico**  $\alpha < 0.5$ .

Ciò contrasta con il separate chaining, dove in genere  $\alpha > 1$ .

# Hashing con Linear Probing

Come calcoliamo l'hash di una chiave  $k$ ?

se la cella  $h(k)$  dell'array  $B$  è **vuota**, inseriamo il record relativo a  $k$  in  $B[h(k)]$ .

Successivamente, se un'altra chiave  $k'$  ha lo **stesso valore di hash** di  $k$ , cioè  $h(k') = h(k)$ , non possiamo inserire il record di  $k'$  in  $B[h(k)]$ .

In tal caso procediamo **cella per cella**, a partire da  $B[h(k) + 1]$ , poi  $B[h(k) + 2]$ , e così via, **ciclicamente modulo  $n$** , finché non troviamo la **prima cella libera**.

Inseriamo quindi il record di  $k'$  in quella cella.

# Hashing con Linear Probing

## Linear Probing — Inserimento

Inserisci la chiave  $k$  nella **prima cella libera** dell'array  $B$ , procedendo in ordine

$$B[h(k)], B[(h(k) + 1) \bmod n], B[(h(k) + 2) \bmod n], \dots$$

finché si trova una cella vuota.

Il probing è **ciclico modulo  $n$** .

# Hashing con Linear Probing

**Domanda:** Quando cerchiamo una chiave  $k$ , come facciamo a sapere che  $k$  non è nella tabella?

**Risposta:** Iniziamo controllando la cella  $h(k)$ , poi  $h(k) + 1$ , e così via modulo  $n$ , finché non troviamo una **cella vuota**.

La cella vuota è il segnale che la chiave  $k$  **non è presente** nella tabella.

**Domanda:** Ma cosa succede se quella cella vuota è stata creata da una cancellazione?

**Risposta:** Quando una chiave viene cancellata, la cella non viene svuotata: la si marca con un **simbolo speciale**.

Il simbolo indica che la cella **era occupata in passato**, così la ricerca **non si interrompe prematuramente**.

# Hashing con Linear Probing

Il costo delle operazioni di **inserimento** e **cancellazione** può essere limitato superiormente dal costo della ricerca.

Ci riferiamo al **costo di una ricerca non riuscita** — cioè alla ricerca di una chiave **che non è presente** nell'array. Il costo di una ricerca riuscita è infatti **limitato superiormente** da quello di una ricerca non riuscita.

Purtroppo, l'hashing con **linear probing** può generare **clustering**, cioè **lunghe sequenze di celle piene**.

# Hashing con Random Probing

**Come ridurre il clustering?**

**Idea:** rendere il probing **non lineare**.

Per ogni chiave  $k$ , la sequenza di probing è casuale:

$$\langle h(k, 1), h(k, 2), h(k, 3), \dots, h(k, n) \rangle$$

Se  $h(k, i)$  è piena  $\rightarrow$  prova la **prossima cella nella sequenza** (non necessariamente consecutiva)

**Risultato:** celle piene più **uniformemente distribuite**  $\rightarrow$  meno clustering.

# Hashing con Random Probing

**Open Addressing con sequenze di probing uniformi.**

- Ad ogni chiave  $k$  viene associata una **permutazione casuale** delle  $n$  celle  
→ uguale probabilità fra tutte le  $n!$ .
- Per ciascuna chiave  $k$  la sequenza è fissata.

Migliori prestazioni rispetto al **linear probing**

Pur richiedendo salti a posizioni non contigue nell'array, tutti i salti avvengono **all'interno dello stesso array.**

**Risultato:** meno clustering, ricerche più rapide.

# Hashing con Random Probing

## Teorema 20.5

Sia  $m$  il numero di chiavi inserite in una tabella di  $n$  celle tramite **open addressing con sequenze di probing uniformi e casuali**. Sia

$$\alpha = \frac{m}{n} < 1$$

il fattore di carico.

Allora, il **costo atteso di una ricerca non riuscita** è al più

$$\frac{1}{1 - \alpha}.$$

Quando la tabella è poco piena ( $\alpha$  piccolo), la ricerca è molto veloce.

# Hashing con Random Probing

**Dim.** Sia  $X$  il **costo di ricerca non riuscita**.

- $P\{X > 0\} = 1$  (*bisogna sempre sondare almeno una cella*)
- $P\{X > 1\} = P\{\text{la prima cella esaminata è occupata}\} = \frac{m}{n} = \alpha.$

Sia  $A_i$  l'evento che la  $i$ -esima cella esaminata sia **occupata**.

$$\begin{aligned} P\{X > 2\} &= P\{\text{le prime due celle esaminate sono occupate}\} \\ &= P\{A_1 \cap A_2\} = P\{A_1\} \cdot P\{A_2 \mid A_1\}. \end{aligned}$$

Poiché dopo il primo probing restano  $m - 1$  chiavi e  $n - 1$  celle:

$$P\{A_1\} = \alpha, \quad P\{A_2 \mid A_1\} = \frac{m - 1}{n - 1}.$$

# Hashing con Random Probing

Dunque:

$$P\{X > 2\} = \alpha \cdot \frac{m-1}{n-1} = \alpha \cdot \frac{\alpha n - 1}{n-1} < \alpha \cdot \frac{\alpha n}{n} = \alpha^2.$$

Usando la **regola della catena** (Teorema 2.10) abbiamo:

$$P\{X > i\} = P\{\text{le prime } i \text{ celle esaminate sono occupate}\}$$

$$= P\{A_1 \cap A_2 \cap \dots \cap A_i\} = P\{A_1\} P\{A_2 \mid A_1\} \dots P\{A_i \mid A_1 \cap A_2 \cap \dots \cap A_{i-1}\}$$

$$= \frac{m}{n} \cdot \frac{m-1}{n-1} \cdot \frac{m-2}{n-2} \dots \frac{m-i+1}{n-i+1}.$$

# Hashing con Random Probing

Poiché  $\frac{m-j}{n-j} \leq \frac{m}{n} = \alpha$  per ogni  $j$ , otteniamo:

$$P\{X > i\} \leq \alpha^i.$$

La probabilità che servano più di  $i$  probing **decresce esponenzialmente** con  $i$ .

Adesso applichiamo il seguente teorema dimostrato un po' di tempo fa.

**Teorema 4.9: (Definizione Alternativa di Valore Atteso)** Sia  $X$  una v.c. discreta a valori interi non negativi. Allora

$$E[X] = \sum_{x=0}^{\infty} P\{X > x\}.$$

# Hashing con Random Probing

Poiché  $\frac{m-j}{n-j} \leq \frac{m}{n} = \alpha$  per ogni  $j$ , otteniamo:

$$P\{X > i\} \leq \alpha^i.$$

$$E[X] = \sum_{i=0}^{\infty} P\{X > i\} \leq 1 + \sum_{i=1}^{n-1} \alpha^i \leq \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}.$$

È necessaria la convergenza della serie  
che è garantita dall'assunzione  $\alpha < 1$   
per l'open addressing.

# Hashing crittografico

## **Funzioni di hash crittografiche.**

Il loro scopo **non** è la velocità di ricerca: servono a **proteggere (offuscare) informazioni**, ad esempio le **password**.

**Scenario:** servizio accessibile solo a studenti → login con **ID + password**

**Soluzione Naïf:** salvare password in chiaro in un database.

**Inconveniente:** Se il database viene violato → **tutte le password verrebbero esposte.**

⇒ **Necessità:** funzioni di hash crittografiche (verifica senza rivelare la password).

# Hashing crittografico

Supponiamo che l'ID di Mark Stein sia **mstein** e la sua password sia **ILoveToHelp**.

**Domanda:** L'ID di Mark è pubblico. Come possiamo riconoscere Mark tramite il suo ID e la sua password **senza mai memorizzare la password?**

**Risposta:**

La soluzione consiste nell'usare una **funzione di hash crittografica** per trasformare le password in **firme digitali** (*signatures*) e **memorizzare nel database solo le firme**, non le password in chiaro.

# Hashing crittografico

Usando una **funzione di hash crittografica**, applichiamo l'hash alla password ILoveToHelp, ottenendo la firma a 32 bit 0x1b3a4f52, e memorizziamo nel database ``mstein : 0x1b3a4f52''.

| ID     | Firma (hash della password) |
|--------|-----------------------------|
| mstein | 0x1b3a4f52                  |
| tchen  | 0x51c2df33                  |
| dkosh  | 0xbb89e27a                  |
| dkana  | 0x2f85ad73                  |
| ...    | ...                         |

*Nota:* questa tabella **non è una tabella hash**, ma un semplice **database**.  
La cosa importante è che, **guardando il database, non si possa risalire** alle password.

# Hashing crittografico

Supponiamo che Mark tenti di accedere al servizio delle valutazioni dei corsi con ID = mstein e password = ILoveToHelp.

Per verificare la password, il sistema calcola  $h(\text{ILoveToHelp}) = 0x1b3a4f52$ .

| ID     | Firma (hash della password) |
|--------|-----------------------------|
| mstein | 0x1b3a4f52                  |
| tchen  | 0x51c2df33                  |
| dkosh  | 0xbb89e27a                  |
| dkana  | 0x2f85ad73                  |
| ...    | ...                         |

Confrontiamo quindi il valore 0x1b3a4f52 con la firma memorizzata nel database all'ID **mstein**. I due valori coincidono: Mark ha (*probabilmente*) inserito la password corretta.

# Hashing crittografico

## Definizione 20.6 — Funzione di hash crittografica

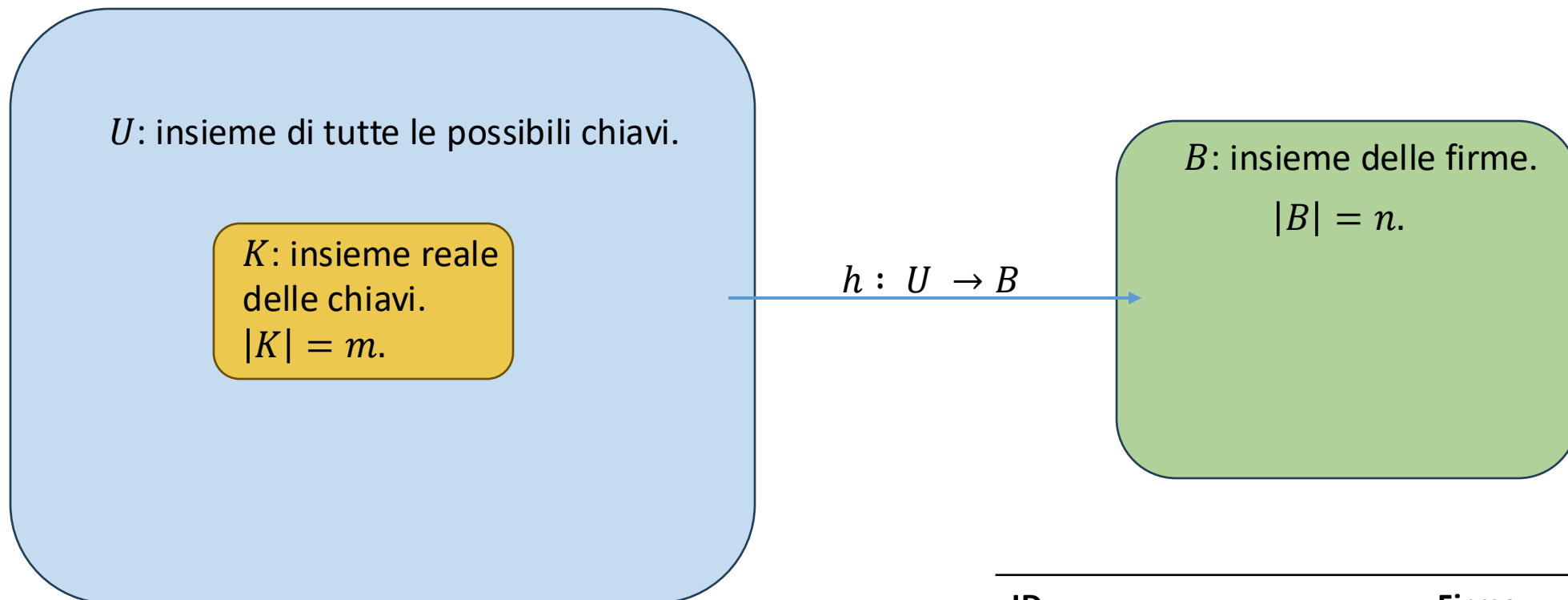
Una **funzione di hash crittografica**  $h : U \rightarrow B$  mappa le **chiavi** in **firme** (*signatures*).

Per una chiave  $k$ , il valore  $h(k)$  è detto **firma di  $k$** . Il **dominio** di  $h$  è  $U$ : l'universo di tutte le possibili chiavi. Il **codominio** è  $B$ : l'insieme di tutte le possibili firme.

$K \subseteq U$  è l'insieme effettivo delle chiavi di cui calcoliamo la firma.

Sia  $|K| = m$  e  $|B| = n$ . In genere vale  $|U| \gg n \gg m$ , perché  $U$  rappresenta un numero potenzialmente infinito di stringhe di qualunque lunghezza, e vogliamo che  $n \gg m$  per **rendere molto improbabili le collisioni**. Di conseguenza, il **fattore di carico**  $\alpha = \frac{m}{n} \ll 1$ .

# Hashing crittografico



Il database contiene  $m$  record (ID e firme)  
Non è una **tabella hash**, ma una semplice  
**mappa ID  $\rightarrow$  firma**

| ID     | Firma      |
|--------|------------|
| mstein | 0x1b3a4f52 |
| tchen  | 0x51c2df33 |
| dkosh  | 0xbb89e27a |
| dkana  | 0x2f85ad73 |
| ...    | ...        |

# Hashing crittografico

## Definizione 20.7 — Collisione di hash.

Una **collisione** si verifica quando **due chiavi diverse** hanno lo **stesso valore di hash**:

$$h(k_1) = h(k_2), \quad \text{con } k_1 \neq k_2.$$

Le collisioni sono indesiderabili (aumentano la vulnerabilità agli attacchi).

Una collisione  $h(k_1) = h(k_2)$  significa che **due password diverse** producono **la stessa firma hash**. Quindi, se nel database la firma salvata per un certo utente è  $s = h(k_1)$ , anche  $k_2$  verrà accettata come valida, perché  $h(k_2) = s$ . Il sistema non distingue tra le due password.

**Ideale:** mappatura uno-a-uno.

**Realtà:** impossibile, perché  $|U| \gg n \rightarrow$  per il *principio dei cassetti*, le collisioni sono inevitabili.

# Hashing crittografico

**Domanda:** Supponiamo che un attaccante provi  $m$  password diverse (chiavi).

Assumiamo SUHA, quindi ogni chiave ha probabilità  $1/n$  di cadere in un qualunque bucket. Qual è la probabilità  $p(m, n)$  che **non** si verifichi alcuna collisione?

Sia  $A$  l'evento in cui **non avvengono collisioni**, cioè nessuna coppia di chiavi ha la stessa firma. Immaginiamo le chiavi ordinate da 1 a  $m$ .

Sia  $A_i$  l'evento in cui la chiave  $i$ -esima ha una firma diversa da tutte le prime  $i - 1$  chiavi. Allora:

$$A = \bigcap_{i=1}^m A_i .$$

# Hashing crittografico

Quindi (regola della catena):

$$\mathbf{P}\{A\} = \mathbf{P}\{A_1\} \cdot \prod_{i=2}^m \mathbf{P}(A_i \mid \cap_{j=1}^{i-1} A_j)$$

La prima chiave non può collidere:  $\mathbf{P}\{A_1\} = 1$ . Per ogni chiave successiva la probabilità di non collidere con le precedenti è:

$$\mathbf{P}(A_i \mid \cap_{j=1}^{i-1} A_j) = 1 - \frac{i-1}{n}.$$

Da cui:

$$\mathbf{P}\{A\} = \prod_{i=2}^m \left(1 - \frac{i-1}{n}\right) = \prod_{i=1}^{m-1} \left(1 - \frac{i}{n}\right).$$

# Hashing crittografico

Ricordando che per  $0 < x < 1$ , vale  $e^{-x} > 1 - x$ :

$$\begin{aligned} P\{A\} &= \prod_{i=1}^{m-1} \left(1 - \frac{i}{n}\right) < \prod_{i=1}^{m-1} e^{-i/n} \\ &= \exp\left(-\frac{1}{n} \sum_{i=1}^{m-1} i\right) = \exp\left(-\frac{m(m-1)}{2n}\right). \end{aligned}$$

La probabilità che **non avvengano collisioni** decresce **esponenzialmente** con  $m^2/n$ .

# Hashing crittografico

## Teorema 20.8 — Probabilità di assenza di collisioni.

Se usiamo una **funzione hash semplice e uniforme** per mappare  $m$  chiavi in uno spazio di hash di dimensione  $n$ , la probabilità che **non si verifichino collisioni** è

$$p(m, n) = \prod_{i=1}^{m-1} \left(1 - \frac{i}{n}\right).$$

Questa è limitata superiormente da  $p(m, n) \leq e^{-\frac{m(m-1)}{2n}}$ .

Assumendo  $n \gg m$ , questo **upper bound** è **molto vicino al valore reale**.

# Hashing crittografico

**Dim.**

L'unico punto che resta da dimostrare è la **bontà dell'approssimazione** (tightness) del bound superiore.

Osserviamo che la disuguaglianza

$1 - \frac{x}{n} \leq e^{-x/n}$  è **quasi un'uguaglianza** quando  $n \gg x$ .

In particolare, se  $n \gg m$ , allora questa approssimazione è **molto accurata per ciascuno** degli  $m$  termini del prodotto che definisce  $p(m, n)$ .